



Computation & Complexity

Chap 3 Undecidability

Yuzhen Niu



3.1 The Church-Turing Thesis

In this book we address this question: What can be computed? (And, more intriguingly, what *cannot* be computed?) We have introduced various and diverse mathematical models of computational processes that accomplish concrete computational tasks—in particular, decide, semidecide, or generate languages, and compute functions.

All this suggests that we have reached a natural upper limit on what a computational device can be designed to do; that our search for the ultimate and most general mathematical notion of a computational process, of an *algorithm*, has been concluded successfully—and the Turing machine is the right answer. However, we have also seen in the last chapter that not all Turing machines deserve to be called “algorithms.” We argued that Turing machines that semidecide languages, and thus reject by never halting, are not useful computational devices, whereas Turing machines that decide languages and compute functions (and therefore halt at all inputs) are. Our notion of an algorithm must exclude Turing machines that may not halt on some inputs.



3.1 The Church-Turing Thesis

We therefore propose to adopt the Turing machine that halts on all inputs as the precise formal notion corresponding to the intuitive notion of an “algorithm.” Nothing will be considered an algorithm if it cannot be rendered as a Turing machine that is guaranteed to halt on all inputs, and all such machines will be rightfully called algorithms. This principle is known as the **Church-Turing thesis**. It is a thesis, not a theorem, because it is not a mathematical result: It simply asserts that a certain informal concept (algorithm) corresponds to a certain mathematical object (Turing machine). Not being a mathematical statement, the Church-Turing thesis cannot be proved. It is theoretically possible, however, that the Church-Turing thesis could be *disproved* at some future date, if someone were to propose an alternative model of computation that was publicly acceptable as a plausible and reasonable model of computation, and yet was provably capable of carrying out computations that cannot be carried out by any Turing machine. No one considers this likely.



3.1 The Church-Turing Thesis

Adopting a precise mathematical notion of an algorithm opens up the intriguing possibility of formally proving that certain computational problems *cannot* be solved by *any* algorithm. We already know enough to expect this. In Chapter 1 we argued that if strings are used to represent languages, not every language can be represented: there are only a countable number of strings over an alphabet, and there are uncountably many languages.

Accordingly, there are only countably many recursive and recursively enumerable languages over any alphabet. So although we have worked hard to extend the capabilities of computing machines as far as possible, in absolute terms they can be used for semideciding or deciding only an infinitesimal fraction of all the possible languages.



3.1 The Church-Turing Thesis

●
according to the Church-Turing thesis, computational tasks that cannot be performed by Turing machines are impossible, hopeless, *undecidable*.

反省的; 内省的
we must devise techniques for exploiting the considerable *power* of Turing machines in order to expose their limitations. The aspect of the power of Turing machines that we will explore is a kind of *introspective* ability they possess: We point out that Turing machines can receive encodings of Turing machines as inputs, and manipulate these encodings in interesting ways. We will then ask what happens when a Turing machine manipulates an encoding of itself—an ingenious yet simple application of the diagonalization principle. How to encode a Turing machine so it can be manipulated by another (or *the same!*) Turing machine is thus our next subject.



3.2 Universal Turing Machines

Software or Hardware

- Turing machine we studied so far solve a specific problem.
- It can not be programmed for different tasks.
- So they are "hard-wired at the factory".
- This kind of Turing machine acts as hardware.

Software or Hardware

- Is that possible Turing machines can be programmed?
- There is a certain "generic" Turing machine that can be programmed.
- The program that makes this machine behave like a specific machine "M" will have to be a "description of M".
- We can think of the Turing machine as "programming language".
- Programs written in this language can then be interpreted by a "Universal Turing machine", i.e.,
- Another program in the same language.



3.2 Universal Turing Machines

- we shall show that there is a certain “generic” Turing machine that can be programmed, about the same way that a general-purpose computer can, to solve *any* problem that can be solved by Turing machines. The “program” that makes this generic machine behave like a specific machine M will have to be *a description of M* . In other words, we shall be thinking of the formalism of Turing machines as a *programming language*, in which we can write programs. Programs written in this language can then be *interpreted* by a *universal Turing machine* —that is to say, another program in the same language.



3.2 Universal Turing Machines

- To begin, we must present a general way of specifying Turing machines, so that their descriptions can be used as input to other Turing machines. That is, we must define a language whose strings are all legal representations of Turing machines. One problem manifests itself already: No matter how large an alphabet we choose for this representation, there will be Turing machines that have more states and more tape symbols. Evidently, we must encode the states and tape symbols as strings over a fixed alphabet. We adopt the following convention: A string representing a Turing machine state is of the form $\{q\}\{0, 1\}^*$; that is, the letter q followed by a binary string. Similarly, a tape symbol is always represented as a string in $\{a\}\{0, 1\}^*$.



3.2 Universal Turing Machines

- Let $M = (K, \Sigma, \delta, s, H)$ be a Turing machine, and let i and j be the smallest integers such that $2^i \geq |K|$, and $2^j \geq |\Sigma| + 2$. Then each state in K will be represented as a q followed by a binary string of length i ; each symbol in Σ will be likewise represented as the letter a followed by a string of j bits. The head directions $\leftarrow$ and $\rightarrow$ will also be treated as “honorary tape symbols” (they were the reason for the “+2” term in the definition of j). We fix the representations of the special symbols $\sqcup, \triangleright, \leftarrow$, and $\rightarrow$ to be the lexicographically four smallest symbols, respectively: $\sqcup$ will always be represented as $a0^j$, $\triangleright$ as $a0^{j-1}1$, $\leftarrow$ as $a0^{j-2}10$, and $\rightarrow$ as $a0^{j-2}11$. The start state will always be represented as the lexicographically first state, $q0^i$. Notice that we require the use of leading zeros in the strings that follow the symbols a and q , to bring the total length to the required level.



3.2 Universal Turing Machines

[M]

We shall denote the representation of the whole Turing machine M as “ M ”. “ M ”, consists of the transition table δ . That is, it is a sequence of strings of the form (q, a, p, b) , with q and p representations of states and a, b of symbols, separated by commas and included in parentheses. We adopt the convention that the quadruples are listed in *increasing lexicographic order*, starting with $\delta(s, \sqcup)$. The set of halting states H will be determined indirectly, by the absence of its states as first components in any quadruple of “ M ”. If M decides a language, and thus $H = \{y, n\}$, we will adopt the convention that y is the lexicographically smallest of the two halt states.

This way, any Turing machine can be represented. We shall use the same method to represent *strings* in the alphabet of the Turing machine. Any string $w \in \Sigma^*$ will have a unique representation, also denoted “ w ”, namely, the juxtaposition of the representations of its symbols.



3.2 Universal Turing Machines

Example 5.2.1: Consider the Turing machine $M = (K, \Sigma, \delta, s, \{h\})$, where $K = \{s, q, h\}$, $\Sigma = \{\sqcup, \triangleright, a\}$, and δ is given in this table.

state, symbol		δ
s	a	$(q, \sqcup)$
s	$\sqcup$	$(h, \sqcup)$
s	$\triangleright$	$(s, \rightarrow)$
q	a	(s, a)
q	$\sqcup$	$(s, \rightarrow)$
q	$\triangleright$	$(q, \rightarrow)$



3.2 Universal Turing Machines

Example 5.2.1: Consider the Turing machine $M = (K, \Sigma, \delta, s, \{h\})$, where $K = \{s, q, h\}$, $\Sigma = \{\sqcup, \triangleright, a\}$, and δ is given in this table.

state,	symbol	δ
s	a	$(q, \sqcup)$
s	$\sqcup$	$(h, \sqcup)$
s	$\triangleright$	$(s, \rightarrow)$
q	a	(s, a)
q	$\sqcup$	$(s, \rightarrow)$
q	$\triangleright$	$(q, \rightarrow)$

state/symbol	representation
s	$q00$
q	$q01$
h	$q11$
$\sqcup$	$a000$
$\triangleright$	$a001$
$\leftarrow$	$a010$
$\rightarrow$	$a011$
a	$a100$

Since there are three states in K and three symbols in Σ , we have $i = 2$ and $j = 3$. These are the smallest integers such that $2^i \geq 3$ and $2^j \geq 3 + 2$. The states and symbols are represented as follows:



3.2 Universal Turing Machines

state,	symbol	δ
s	a	$(q, \sqcup)$
s	$\sqcup$	$(h, \sqcup)$
s	$\triangleright$	$(s, \rightarrow)$
q	a	(s, a)
q	$\sqcup$	$(s, \rightarrow)$
q	$\triangleright$	$(q, \rightarrow)$

state/symbol	representation
s	$q00$
q	$q01$
h	$q11$
$\sqcup$	$a000$
$\triangleright$	$a001$
$\leftarrow$	$a010$
$\rightarrow$	$a011$
a	$a100$

Thus, the representation of the string $\triangleright aa \sqcup a$ is

$$“\triangleright aa \sqcup a” = a001a100a100a000a100.$$

The representation “ M ” of the Turing machine M is the following string:

$$“M” = (q00, a100, q01, a000), (q00, a000, q11, a000), (q00, a001, q00, a011), (q01, a100, q00, a011), (q01, a000, q00, a011), (q01, a001, q01, 011). \blacklozenge$$



3.2 Universal Turing Machines

Now we are ready to discuss a *universal Turing machine* U , which uses the encodings of other machines as programs to direct its operation. Intuitively, U takes two arguments, a description of a machine M , “ M ”, and a description of an input string w , “ w ”. We want U to have the following property: U halts on input “ M ” “ w ” if and only if M halts on input w . To use the functional notation for Turing machines we developed in the last chapter,

$$U(\text{“}M\text{” “}w\text{”}) = \text{“}M(w)\text{”}.$$

$$U([M], [w]_M) = [M(w)]_M$$



3.2 Universal Turing Machines

We actually describe not the single-tape machine U , but a closely related 3-tape machine U' (then U will be the single-tape Turing machine that simulates U'). Specifically, U' uses its three tapes as follows: the first tape contains the encoding of the current tape contents of M ; the second tape contains the encoding of M itself; and the third tape contains the encoding of the state of M at the current point in the simulated computation.

The machine U' is started with some string “ M ” “ w ” on its first tape and the other two tapes blank. (It does not matter how U' behaves if its input string is not of this form.) First U' moves “ M ” onto the second tape and shifts “ w ” down to the left end of the first tape, preceding it by “ $\triangleright \sqcup$ ”. Thus at this point the first tape contains “ $\triangleright \sqcup w$ ”. U' writes in the third tape the encoding of the initial state s of M , always $q0^i$ (U' can easily determine i and j by examining “ M ”). Now U' sets about simulating the steps of the computation of M . Between such simulated steps, U' will keep the heads on the second and third tapes at their left ends, and the head of the first tape scanning the a of the encoded version of the symbol that M would be scanning at the corresponding time.



3.2 Universal Turing Machines

U' simulates a step of M as follows: It scans its second tape until it finds a quadruple whose first component matches the encoded state written in its third tape, and whose second component matches the encoded symbol scanned in the first tape. If it finds such a quadruple, it changes the state to the third component of that quadruple, and performs in the first tape the action suggested by the fourth component. If the fourth component encodes a symbol of the tape alphabet of M , this symbol is written in the first tape. If the fourth component is $a0^{j-2}10$, the encoding of $\leftarrow$, then U' moves its first head to the first a symbol to the left, and if it is the encoding of $\rightarrow$, to the right. If a $\sqcup$ is encountered, U' must convert it to $a0^j$, the encoding of a blank of M .

If at some step the state-symbol combination is not found in the second tape, this means that the state is a halting state. U' also halts at an appropriate state. This completes our description of the operation of U' .

Interpretation of Turing Machine

- We need encoding of the Turing machine to feed it as the input of another Turing machine.
- We need to define a language whose strings are all legal representation of Turing machines.

Description Number

- A string representing a state of a Turing machine is of the form $\{q\}\{0,1\}^*$.
- Similarly, a string representing a tape symbol is of the form $\{a\}\{0,1\}^*$.

Description Number

- Let $M = (Q, \Sigma, \delta, s, H)$ be a Turing machine.
- Let i and j be the smallest numbers such that $|Q| \leq 2^i$ and $|\Sigma| + 2 \leq 2^j$
- Then if $p \in Q$ then representation of p is $[p]_M = q\{0, 1\}^i$.
- If $b \in \Sigma$ then representation of b is $[b]_M = a\{0, 1\}^j$.

Description Number

- $[\sqcup]_M = a0^j$
- $[\triangleright] = a0^{j-1}1$
- $[\mathbf{L}] = a0^{j-2}10$
- $[\mathbf{R}] = a0^{j-2}1$
- $[s] = q0^i$

Description Number

- If $\delta(q, a) = (p, b)$ then we write this transition as (q, a, p, b) and by coding this 4-tuple we have encoding of this transition.
- These 4-tuples are listed in increasing lexicographic order.
- It always start with $\delta(s, \sqcup)$.
- The set of halting states will be determined indirectly as follow,
- By absence of its states as first components in any quardruple of M .
- If $H = \{y, n\}$ we consider y as the smaller one.
- We denote the representation of the whole Turing machine as $[M]$

Example

- Let $M = (Q, \Sigma, \delta, s, \{h\})$ where $Q = \{s, q, h\}$, $\Sigma = \{\sqcup, \triangleright, a\}$ and δ is as follow;

$$\delta(s, a) = (q, \sqcup)$$

$$\delta(s, \sqcup) = (h, \sqcup)$$

$$\delta(s, \triangleright) = (s, \mathbf{R})$$

$$\delta(q, a) = (s, a)$$

$$\delta(q, \sqcup) = (s, \mathbf{R})$$

$$\delta(q, \triangleright) = (q, \mathbf{R})$$

Example

- Since $|Q| = 3$ and $|\Sigma| = 3$ we need to find the smallest numbers i and j such that, $3 \leq 2^i$ and $3 + 2 \leq 2^j$.
- Thus $i = 2$ and $j = 3$.
- $[s]_M = q00$
- $[q]_M = q01$
- $[h]_M = q11$

Example

- $[\sqcup]_M = a000$
- $[\triangleright]_M = a001$
- $[\mathbf{L}]_M = a010$
- $[\mathbf{R}]_M = a011$
- $[a]_M = a100$.
- For example:

$$\begin{aligned}[\triangleright \sqcup aa]_M &= [\triangleright]_M [\sqcup]_M [a]_M [a]_M \\ &= a001a000a100a100\end{aligned}$$

Example

- Representation of M is as follow;

$$\begin{aligned}[M] = & (q00, a100, q01, a000), (q00, a000, q11, a000), \\ & (q00, a001, q00, a011), (q01, a100, q00, a011), \\ & (q01, a000, q00, a011), (q01, a001, q01, 011)\end{aligned}$$

Universal Turing Machine

- A **universal Turing machine** U is a Turing machine such that;
- Uses encoding of other Turing machines as programs to direct its operations.
- U takes two arguments, a description of a Turing machine M , $[M]$ and a description of an input string w , $[w]_M$.
- U halts on $([M], [w]_M)$ if and only if M halts on w .
- So, $U([M], [w]_M) = [M(w)]_M$



3.3 The Halting Problem

Suppose that you have written a program, in your favorite programming language, that performs the following remarkable feat: It takes as input any program P , written in the same language, and an input X of that program. By some clever analysis, your program always determines correctly whether the program P would halt on input X (it returns “yes” if it does), or whether it would run forever (it returns “no”). You have named this program $\text{halts}(P, X)$.

This is a most valuable program. It discovers all sorts of subtle bugs that make other programs run forever on certain inputs. Using it you can achieve many remarkable things.



3.3 The Halting Problem

- You can use it to write another program, with the ominous name $\text{diagonal}(X)$:

$\text{diagonal}(X)$

a : if $\text{halts}(X, X)$ then goto a else halt

Notice what $\text{diagonal}(X)$ does: If your halts program decides that program X would halt if presented with *itself* as input, then $\text{diagonal}(X)$ loops forever; otherwise it halts.

And now comes the unanswerable question: *Does $\text{diagonal}(\text{diagonal})$ halt?* It halts if and only if the call $\text{halts}(\text{diagonal}, \text{diagonal})$ returns “no”; in other words, *it halts if and only if it does not halt*. This is a contradiction: we must conclude that the only hypothesis that started us on this path is false, that program $\text{halts}(P, X)$ does not exist. That is to say, there can be no program, no algorithm for solving the problem halts would solve: to tell whether arbitrary programs would halt or loop.

Halting Problem

- Suppose **halts** be a universal Turing machine such that;
- If program P halts on input X then **halts**(P, X) = *yes* and if it doesn't **halts**(P, X) = *no*.
- Now define **diagonal**(X) as follow;
- If **halts**(X, X) = *yes* then **diagonal**(X) loops forever otherwise halts.

Halting Problem

- Does **diagonal(diagonal)** halt?
- If it halts then **halts(diagonal, diagonal)** = *no*, i.e.,
- **diagonal(diagonal)** does not halt, contradiction!
- So **halts(P, X)** does not exist.



3.3 The Halting Problem

We are thus ready to define a language that is not recursive, and *prove* that it is not. Let

$$\text{Halt}_{\text{TM}} = \{ \langle M, w \rangle \mid \text{TM } M \text{ halts on input string } w \}.$$

$$H = \{ \text{"}M\text{" "}w\text{"} : \text{Turing machine } M \text{ halts on input string } w \}.$$

Notice first that H is recursively enumerable: It is precisely the language semidecided by our universal Turing machine U in the previous section. Indeed, on input $\text{"}M\text{" "}w\text{"}$, U halts precisely when the input is in H .

Furthermore, if H is recursive, then *every* recursively enumerable language is recursive. In other words, H holds the key to the question we asked in Section 4.2, whether all recursively enumerable languages are also Turing decidable: the answer is positive if and only if H is recursive.

A similar problem:
the acceptance problem

Let $A_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}.$
THEOREM: A_{TM} is undecidable.



3.3 The Halting Problem

Furthermore, if H is recursive, then *every* recursively enumerable language is recursive. In other words, H holds the key to the question we asked in Section 4.2, whether all recursively enumerable languages are also Turing decidable: the answer is positive if and only if H is recursive. For suppose that H is indeed decided by some Turing machine M_0 . Then given any particular Turing machine M semideciding a language $L(M)$, we could design a Turing machine M' that *decides* $L(M)$ as follows: First, M' transforms its input tape from $\triangleright \sqcup w \sqcup$ to $\triangleright "M" "w" \sqcup$ and then simulates M_0 on this input. By hypothesis, M_0 will correctly decide whether or not M accepts w .

假设 H 是decidable, M_0 ;

给定一个TM M semideciding a language $L(M)$;

可以设计 M' decide $L(M)$:

M' 将输入改为 M_0 的输入格式“ M ”“ w ”, 然后模拟 M_0 对输入“ M ”“ w ”进行计算, 由于 M_0 可以decide H , 因此可以decide M 是否接收 w , 将 M 的yes/no作为 M' 的yes/no.即 M' 可以decide $L(M)$.

即如果 H 是recursive, 所有的recursively enumerable language is recursive.



3.3 The Halting Problem

But we can show, by formalizing the argument for $\text{halts}(P, X)$ above, that H is *not* recursive. First, if H were recursive, then

$$H_1 = \{ "M" : \text{Turing machine } M \text{ halts on input string } "M" \}$$

would also be recursive. (H_1 stands for the $\text{halts}(X, X)$ part of the diagonal program.) If there were a Turing machine M_0 that could decide H , then a Turing machine M_1 to decide H_1 would only need to transform its input string $\triangleright \sqcup "M" \sqcup$ to $\triangleright \sqcup "M" "M" \sqcup$ and then yield control to M_0 . Therefore, it suffices to show that H_1 is not recursive.

如果H是recursive, 则H1是recursive(模拟M0对输入“M”“M”进行计算)。
只要证明H1不是recursive, 就能证明H不是recursive。



3.3 The Halting Problem

Second, if H_1 , were recursive, then its complement would also be recursive:

$\overline{H_1} = \{w : \text{either } w \text{ is not the encoding of a Turing machine, or it is the encoding "M" of a Turing machine } M \text{ that does not halt on "M"}\}.$

This is so because the class of recursive languages is closed under complement (Theorem 4.2.2). Incidentally, $\overline{H_1}$ is the *diagonal language*, the analog of our diagonal program, and the last act of the proof.

But $\overline{H_1}$ cannot even be recursively enumerable —let alone recursive. For suppose M^* were a Turing machine that semidecided $\overline{H_1}$. Is " M^* " in $\overline{H_1}$? By definition of $\overline{H_1}$, " M^* " $\in \overline{H_1}$ if and only if M^* does not accept input string " M^* ". But M^* is supposed to semidecide $\overline{H_1}$, so " M^* " $\in \overline{H_1}$ if and only if M^* accepts " M^* ". We have concluded that M^* accepts " M^* " if and only if M^* does not accept " M^* ". This is absurd, so the assumption that M_0 exists must have been in error.

H1 是 recursive 的，则 H1 补也是 recursive 的。但是 H1 补不是 recursively enumerable 的，更不是 recursive 的。假设 H1 补是 recursively enumerable 的，图灵机 M^* semidecide H1 补。由 H1 补的定义和 M^* semidecide H1 补，得出矛盾。

因此假设错误，即 H1 补不是 recursively enumerable 的，更不是 recursive 的 $\rightarrow$ H1 不是 recursive 的 $\rightarrow$ H 不是 recursive 的



3.3 The Halting Problem

Let us summarize the development of this section. We wanted to discover whether every recursively enumerable language is recursive. We observed that this would be true if and only if the particular recursively enumerable language H were recursive. From H we derived, in two steps, the language $\overline{H_1}$, which has to be recursive in order for H to be recursive. But the assumption that $\overline{H_1}$ is recursive led to a logical contradiction, by diagonalization. We have therefore proved the following most important theorem.

Theorem 5.3.1: *The language H is not recursive; therefore, the class of recursive languages is a strict subset of the class of recursively enumerable languages.*



3.3 The Halting Problem

•

It is easy to see that H_1 , like H , is recursively enumerable, and we have shown that $\overline{H_1}$ is not recursively enumerable. Therefore we have also proved the following result.

Theorem 5.3.2: *The class of recursively enumerable languages is not closed under complement.*



3.3 The A_{TM} Problem (The acceptance problem)

- Proof from another perspective.

Let $A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}$.

THEOREM: A_{TM} is undecidable.

- The proof of the undecidability of the acceptance problem uses a technique called **diagonalization**, discovered by mathematician Georg Cantor in 1873.



The Diagonalization Method

- The set of real numbers is an example of an uncountable set. A real number is one that has a decimal representation. The numbers $\pi = 3.1415926\dots$, and $\sqrt{2} = 1.4142135\dots$ are examples of real numbers. Let R be the set of real numbers. Cantor proved that R is **uncountable**. In doing so he introduced the diagonalization method.



The Diagonalization Method

- THEOREM: R is uncountable.
- PROOF:
 - In order to show that R is uncountable, we show that **no correspondence exists between N and R** . The proof is by contradiction. Suppose that a correspondence f existed between N and R . Our job is to show that f fails to work as it should. For it to be a correspondence, f must pair all the members of N with all the members of R . **But we will find an x in R that is not paired with anything in N** , which will be our contradiction.



The Diagonalization Method

- The way we find this x is by actually constructing it. We choose each digit of x to make x different from one of the real numbers that is paired with an element of N . In the end we are sure that x is different from any real number that is paired.

n	$f(n)$	
1	3. <u>1</u> 4159...	
2	55.55 <u>5</u> 55...	
3	0.123 <u>4</u> 5...	$x = 0.4641 \dots$
4	0.500 <u>0</u> 0...	
$\vdots$	$\vdots$	



The A_{TM} Is Undecidable

Let $A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}$.

THEOREM: A_{TM} is undecidable.

- Proof: We assume that A_{TM} is decidable and obtain a contradiction. Suppose that H is a decider for A_{TM} . On input (M, w) , where M is a TM and w is a string, H halts and accepts if M accepts w . Furthermore, H halts and rejects if M fails to accept w . In other words, we assume that H is a TM, where

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w \\ \text{reject} & \text{if } M \text{ does not accept } w. \end{cases}$$



The A_{TM} Is Undecidable

- Now we **construct a new Turing machine D with H as a subroutine**. This new TM calls H to determine what M does when the input to M is its own description $\langle M \rangle$. Once D has determined this information, it **does the opposite**. That is, it rejects if M accepts and accepts if M does not accept. The following is a description of D.

$D =$ “On input $\langle M \rangle$, where M is a TM:

1. Run H on input $\langle M, \langle M \rangle \rangle$.
2. Output the opposite of what H outputs; that is, if H accepts, *reject* and if H rejects, *accept*.”



The A_{TM} Is Undecidable

- Don't be confused by the idea of running a machine on its own description. That is similar to running a program with itself as input, something that does occasionally occur in practice.
- For example, a compiler is a program that translates other programs. A compiler for the language Pascal may itself be written in Pascal, so running that program on itself would make sense. In summary,

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ does not accept } \langle M \rangle \\ \text{reject} & \text{if } M \text{ accepts } \langle M \rangle. \end{cases}$$



The A_{TM} Is Undecidable

- What happens when we run D with its own description(D) as input? In that case we get

$$D(\langle D \rangle) = \begin{cases} \text{accept} & \text{if } D \text{ does not accept } \langle D \rangle \\ \text{reject} & \text{if } D \text{ accepts } \langle D \rangle. \end{cases}$$

- No matter what D does, it is forced to do the opposite, which is obviously a contradiction. Thus neither TM D nor TM H can exist.



The A_{TM} Is Undecidable

- Lets review the steps of this proof. Assume that a TM H decides A_{TM} . Then use H to build a TM D that when given input $\langle M \rangle$ accepts exactly when M does not accept input $\langle M \rangle$. Finally, run D on itself. The machines take the following actions, with the last line being the contradiction.
 - H accepts $\langle M, w \rangle$ exactly when M accepts w .
 - D rejects $\langle M \rangle$ exactly when M accepts $\langle M \rangle$.
 - D rejects $\langle D \rangle$ exactly when D accepts $\langle D \rangle$.



The A_{TM} Is Undecidable

Where is the diagonalization in the proof of this Theorem? It becomes apparent when you examine tables of behavior for TMs H and D . In these tables we list all TMs down the rows, $M_1, M_2, \dots$ and all their descriptions across the columns, $\langle M_1 \rangle, \langle M_2 \rangle, \dots$. **The entries tell whether the machine in a given row accepts the input in a given column.** The entry is *accept* if the machine accepts the input but is blank if it rejects or loops on that input. We made up the entries in the following figure to illustrate the idea.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\dots$
M_1	<i>accept</i>		<i>accept</i>		
M_2	<i>accept</i>	<i>accept</i>	<i>accept</i>	<i>accept</i>	
M_3					$\dots$
M_4	<i>accept</i>	<i>accept</i>			
$\vdots$			$\vdots$		

FIGURE 4.19

Entry i, j is *accept* if M_i accepts $\langle M_j \rangle$



The A_{TM} Is Undecidable

- In the following figure the entries are the results of running H on inputs corresponding to Figure 4.19. So if M_3 does not accept input $\langle M_2 \rangle$, the entry for row M_3 and column $\langle M_2 \rangle$ is reject because H rejects input $\langle M_3 \langle M_2 \rangle \rangle$.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	accept	reject	accept	reject	
M_2	accept	accept	accept	accept	
M_3	reject	reject	reject	reject	...
M_4	accept	accept	reject	reject	
$\vdots$			$\vdots$		

FIGURE 4.20

Entry i, j is the value of H on input $\langle M_i, \langle M_j \rangle \rangle$



The A_{TM} Is Undecidable

- In the following figure, we added D to Figure 4.20. By our assumption, H is a TM and so is D . Therefore it must occur on the list $M_1, M_2, \dots$ of all TMs. Note that **D computes the opposite of the diagonal entries**. The contradiction occurs at the point of the question mark where the entry must be the opposite of itself.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\dots$	$\langle D \rangle$	$\dots$
M_1	<u>accept</u>	reject	accept	reject		accept	
M_2	accept	<u>accept</u>	accept	accept	$\dots$	accept	$\dots$
M_3	reject	reject	<u>reject</u>	reject		reject	
M_4	accept	accept	reject	<u>reject</u>		accept	
$\vdots$			$\vdots$		$\ddots$		
D	reject	reject	accept	accept		<u>?</u>	
$\vdots$			$\vdots$				$\ddots$

FIGURE 4.21

If D is in the figure, a contradiction occurs at “?”



The A_{TM} Is Turing-Recognizable

- The following Turing machine U recognizes A_{TM} .

$U =$ “On input $\langle M, w \rangle$, where M is a TM and w is a string:

1. Simulate M on input w .
2. If M ever enters its accept state, *accept*; if M ever enters its reject state, *reject*.”

- Note that this machine loops on input (M, w) if M loops on w , which is why this machine does not decide A_{TM} . If the algorithm had some way to determine that M was not halting on w , it could reject. Hence A_{TM} is sometimes called the halting problem. As we demonstrate, an algorithm has no way to make this determination.



A Turing-Unrecognizable Language

- In the preceding section we demonstrated a language—namely, A_{TM} —that is un-decidable. Now we demonstrate a language that isn't even Turing-recognizable. Note that A_{TM} will not suffice for this purpose because we showed that A_{TM} is Turing-recognizable.
- The following theorem shows that, if both a language and its complement are Turing-recognizable, the language is decidable. Hence, for any undecidable language, either it or its complement is not Turing-recognizable.
- Recall that the complement of a language is the language consisting of all strings that are not in the language. We say that a language is co-Turing-recognizable if it is the complement of a Turing-recognizable language.



A Turing-Unrecognizable Language

- THEOREM 4.22
- A language is decidable iff it is Turing-recognizable and co-Turing-recognizable.
- In other words, a language is decidable exactly when both **it and its complement are Turing-recognizable.**



A Turing-Unrecognizable Language

PROOF We have two directions to prove. First, if A is decidable, we can easily see that both A and its complement $\bar{A}$ are Turing-recognizable. Any decidable language is Turing-recognizable, and the complement of a decidable language also is decidable.

For the other direction, if both A and $\bar{A}$ are Turing-recognizable, we let M_1 be the recognizer for A and M_2 be the recognizer for $\bar{A}$. The following Turing machine M is a decider for A .

$M =$ “On input w :

1. Run both M_1 and M_2 on input w in parallel.
2. If M_1 accepts, *accept*; if M_2 accepts, *reject*.”

Running the two machines in parallel means that M has two tapes, one for simulating M_1 and the other for simulating M_2 . In this case M takes turns simulating one step of each machine, which continues until one of them accepts.



A Turing-Unrecognizable Language

Now we show that M decides A . Every string w is either in A or $\bar{A}$. Therefore either M_1 or M_2 must accept w . Because M halts whenever M_1 or M_2 accepts, M always halts and so it is a decider. Furthermore, it accepts all strings in A and rejects all strings not in A . So M is a decider for A , and thus A is decidable.



A Turing-Unrecognizable Language

COROLLARY 4.23

$\overline{A_{TM}}$ is not Turing-recognizable.

PROOF We know that A_{TM} is Turing-recognizable. If $\overline{A_{TM}}$ also were Turing-recognizable, A_{TM} would be decidable. Theorem 4.11 tells us that A_{TM} is not decidable, so $\overline{A_{TM}}$ must not be Turing-recognizable.

.....



• Thanks !